Data Manipulation Language

- INSERT
- UPDATE
- DELETE
- RENAME

1. INSERT Command

follow same order as table

```
INSERT INTO table_name (col1, col2, ... col3)
        VALUES (val_col1, val_col2, val_col3);

INSERT INTO table_name (c1, c2, c3 ..)
        VALUES (1, 2, 3),
        VALUES (4, 5, 6);
```

multiple row insertions.

UPDATE

id	name	name2	name3
1	A		
2	B → C		

```
UPDATE <table_name>
    SET name = 'c'
    WHERE id = 2;
```

for multiple columns →

```
UPDATE <table_name>
    SET name2 = "C"
        name3 = "B"
    WHERE id = 2;
```

- We can update all rows at once. eg update current value of Semester attribute by 1.

↓

Update Student SET Semester = Semester + 1;

DELETE

- DELETE FROM <table_name>
  WHERE col = value

- DELETE FROM <table_name>
  (All rows deleted)

Delete in Safe_Mode:

DELETE FROM <t_name> WHERE key = value;

Just type it, it enables safe mode

DELETE FROM Insurance_temp1 WHERE year_of_launch = '2020'

⇓

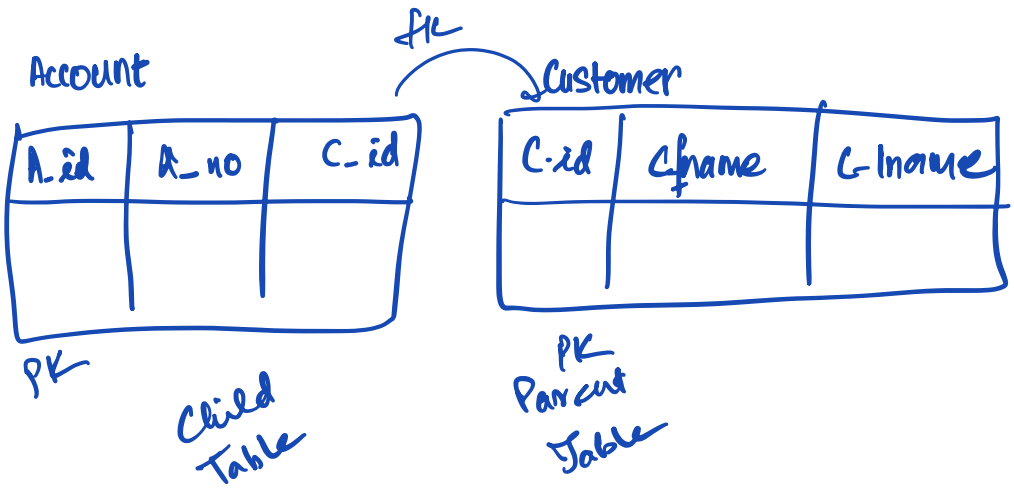
Error: You are using safe update mode & you tried to update w/o a WHERE that uses KEY Column :: ...

to disable it . .

SET SQL - SAFE_UPDATES = 0;

To Enable it = 1

DELETE CASCADE




```
CREATE TABLE ACCOUNT ( id INT, customer_id INT,
                       balance CHAR (50),
                       PRIMARY KEY (id),
                       FOREIGN KEY (customer_id)
                       REFERENCES (customer(id)
                       ON DELETE CASCADE
```

whatever rows delete from parent table also delete from child table.

[Not comma separated]

ON DELETE SET NULL

ON UPDATE CASCADE

REPLACE Command

Case 1 :→ When table has no keys → primary key.

→ Insert a new row into table, if duplicate key error occurs.

→ If insertion fails due to a duplicate key, error occurs.

Syntax :→

```
REPLACE INTO insurance (Col1, Col2) VALUES ('  ', '  ')
```

→ Replace can be generally used for update also

```
REPLACE INTO INSURANCE
        SET product_name = "Inner"
            year_of_launch = "2012",
```

- Thus, if no matching value is found with existing data row, then standard INSERT statement is formed.
- If record exists → REPLACE will delete existing data & add new record.

If we have not specified column's value in the SET statement - REPLACE command works here the UPDATE statement as it'll use the already existing default data.

So, if primary key is same - then updated else inserted.

if not primary key - again it'll insert

<u>REPLACE for updating Data</u>

```
REPLACE INTO insurance_temp1
SET
    product_id = 6,
    product_name = 'insure';
```

if we directly do it, then all other values get Null